

Pretrained Models, Transfer Learning & Fine-Tuning

KAUST–Mawhiba IOAI Training

Week 2 · Day 10



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

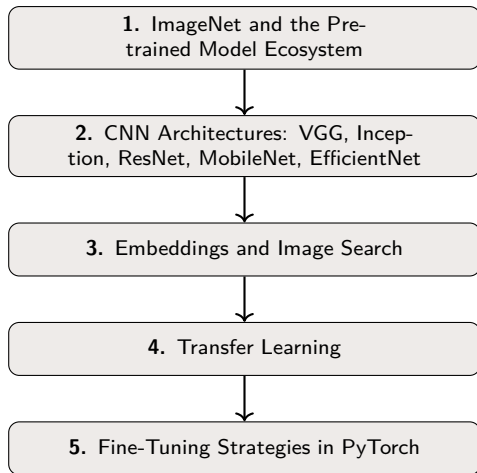
KAUST Academy
King Abdullah University of Science and Technology

May 3, 2026

1. Recap and Roadmap
2. ImageNet and Pretrained Models
3. CNN Architectures: VGG, Inception, ResNet, MobileNet, EfficientNet
4. Embeddings and Image Search
5. Transfer Learning
6. Fine-Tuning Strategies
7. Fine-Tuning in PyTorch
8. Summary

- ▶ **Convolutional layers** extract local patterns (edges, textures, shapes).
- ▶ **Pooling** reduces spatial dimensions while keeping key features.
- ▶ **Data augmentation** creates training variety and reduces overfitting.
- ▶ You built a CNN from scratch and trained it on MNIST.

Training from scratch works — but can we do better?

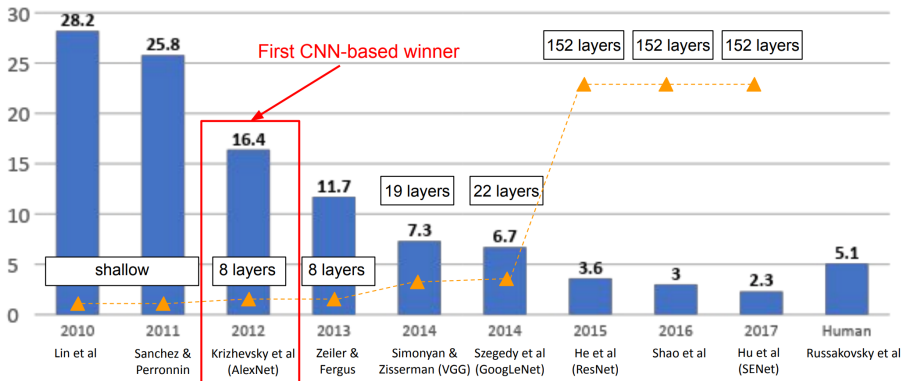


CNN Datasets: **ImageNet**

- ▶ The largest benchmark dataset for image classification.
- ▶ 3 RGB channels, pixel values 0–255.
- ▶ **14,197,122** images across **1000** classes.



ImageNet Large Scale Visual Recognition Challenge (ILSVRC)



- ▶ Training on ImageNet teaches a model to recognize **universal visual features**:
 - Early layers: edges, textures, colors
 - Middle layers: parts (eyes, wheels, leaves)
 - Deep layers: objects (faces, cars, animals)
- ▶ These features **transfer** to new tasks — even tasks the model was never trained on.
- ▶ **Pretrained models** are freely available in PyTorch via `torchvision.models`.
- ▶ Instead of training from scratch, we can **start from a pretrained model** and adapt it.

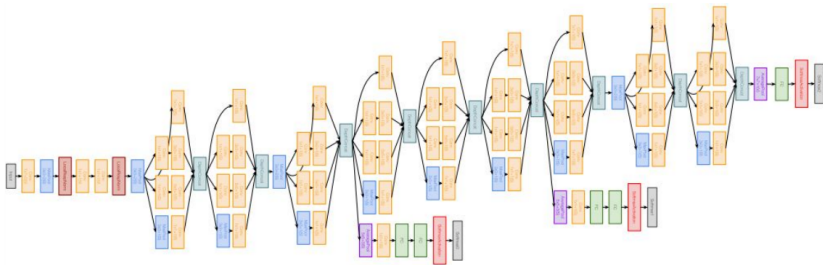
CNN Architectures: **VGGNet**

- **Improvement over AlexNet:** Uses a deeper network with small filters instead of a shallow network with larger filters.
- A stack of 3x3 conv layers (vs. 11x11 conv) has same receptive field, more non-linearities, fewer parameters and deeper network representation.
- Two variants: VGG16 or VGG19 conv layers plus 3 FC layers.

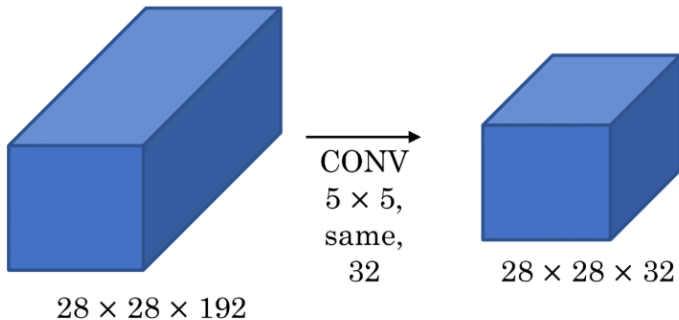


CNN Architectures: **InceptionNet**

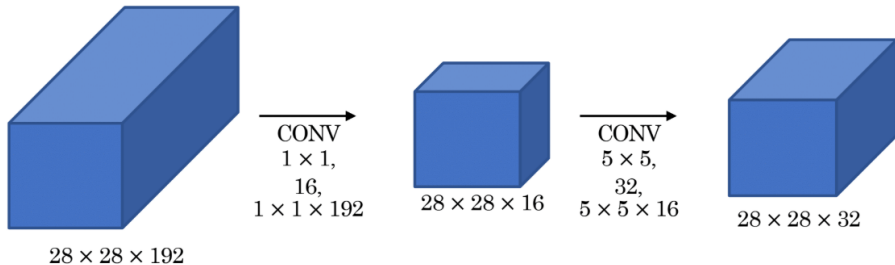
- ▶ InceptionNet goes **deeper** with 22 layers.
- ▶ It is much more **efficient**: only 5 million parameters.
- ▶ Uses a special building block called the **Inception module**.
- ▶ Adds **1x1 convolutions** (bottleneck layers) to make the network faster and smarter.



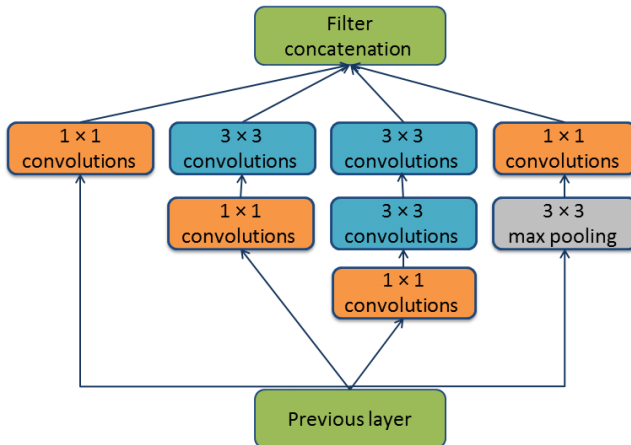
The problem of computational cost

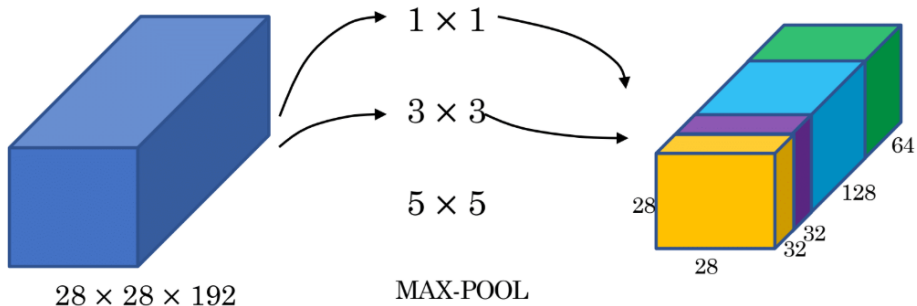


Using 1x1 convolution



- **Inception module:** Uses multiple filter sizes (1×1 , 3×3 , 5×5), in parallel, to capture different features, then combines their outputs.



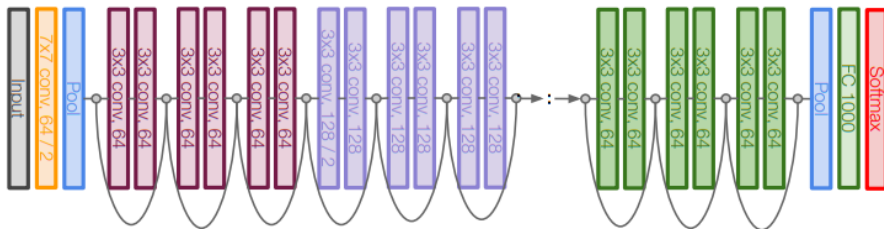


[Szegedy et al. 2014. Going deeper with convolutions]

CNN Architectures: **ResNet**

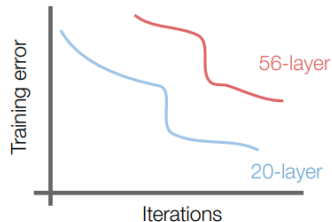
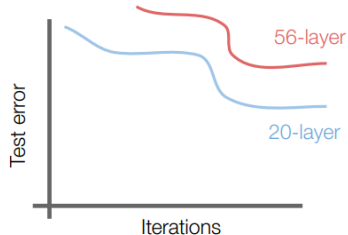
- ▶ **Problem:** Making networks deeper does not always improve accuracy.
- ▶ **Why?** In very deep networks, gradients become extremely small as they move backward through layers, making learning slow or stopping it altogether (**vanishing gradient problem**).
- ▶ **Solution:** Residual Network (ResNet) introduces **skip connections (residuals)**, allowing information to flow more easily.

- ▶ Very deep networks using residual connections
- ▶ 152-layer model for ImageNet
- ▶ Stacked Residual Blocks
- ▶ **Residual:** A shortcut connection that helps the network pass information through layers more easily.

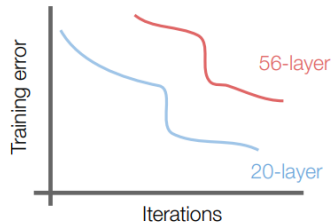
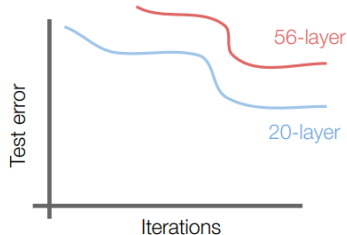


- ▶ What happens when we continue stacking deeper layers on a "plain" convolutional neural network?

- What happens when we continue stacking deeper layers on a "plain" convolutional neural network?

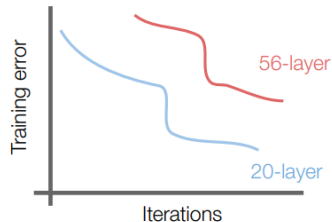
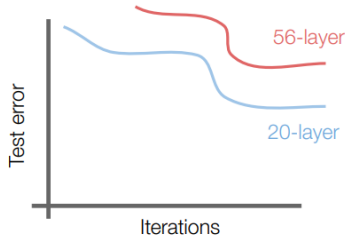


- What happens when we continue stacking deeper layers on a "plain" convolutional neural network?



- 56-layer model performs worse on both test and training error

- ▶ What happens when we continue stacking deeper layers on a "plain" convolutional neural network?



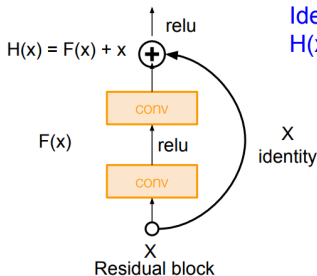
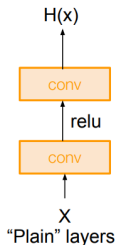
- ▶ 56-layer model performs worse on both test and training error
- ▶ The deeper model performs worse, but it's not caused by overfitting!

- **Fact:** Deep models have more representation power (more parameters) than shallower models.

- ▶ **Fact:** Deep models have more representation power (more parameters) than shallower models.
- ▶ **Hypothesis:** The problem is an optimization problem, deeper models are harder to optimize

- ▶ **Fact:** Deep models have more representation power (more parameters) than shallower models.
- ▶ **Hypothesis:** The problem is an optimization problem, deeper models are harder to optimize
- ▶ **Solution:** Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping

- **Fact:** Deep models have more representation power (more parameters) than shallower models.
- **Hypothesis:** The problem is an optimization problem, deeper models are harder to optimize
- **Solution:** Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping



Identity mapping:
 $H(x) = x$ if $F(x) = 0$

CNN Architectures: **MobileNet**

► Why MobileNets?

- Small-sized models are crucial for mobile and embedded devices.
- MobileNets reduce computational cost and memory usage while maintaining good accuracy.

► Key Idea:

- Use **depthwise-separable convolutions** to significantly reduce computation compared to standard convolutions.

► **Computational cost of standard convolution:**

$$\text{Cost} = \# \text{ filter params} \times \# \text{ filter positions} \times \# \text{ filters}$$

- Filters operate on all input channels, increasing computation significantly.



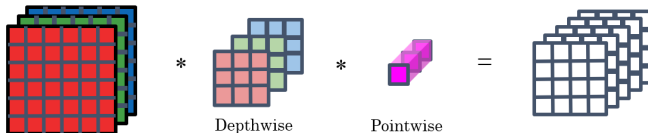
Depthwise-Separable Convolutions

- ▶ Split standard convolution into two steps:
 - **Depthwise Convolution:** Applies a single filter per input channel.
 - **Pointwise Convolution:** Combines outputs from depthwise convolution.
- ▶ **Key Benefit:** Reduces computational cost significantly compared to standard convolution.

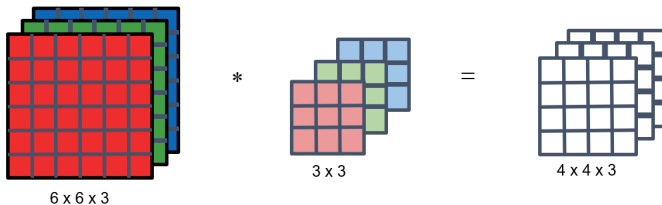
Normal Convolution



Depthwise Separable Convolution

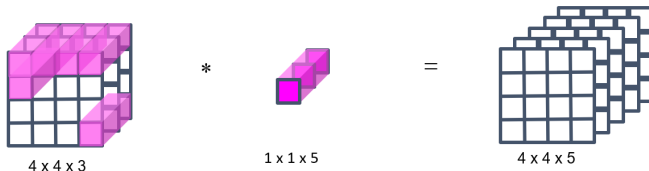


- Operates on each input channel separately.



$$\text{Computational cost} = \# \text{filter params} \times \# \text{filter positions} \times \# \text{of filters}$$

- Combines outputs from depthwise convolution using 1×1 convolutions (mixes channels).



$$\text{Computational cost} = \# \text{filter params} \times \# \text{filter positions} \times \# \text{of filters}$$

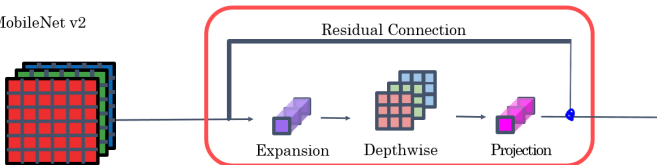
► MobileNet v2:

- Adds **residual connections**.
- Introduces:
 - **Expansion step**: Expands input dimensions before depthwise convolution.
 - **Projection step**: Reduces dimensions after processing.

MobileNet v1



MobileNet v2

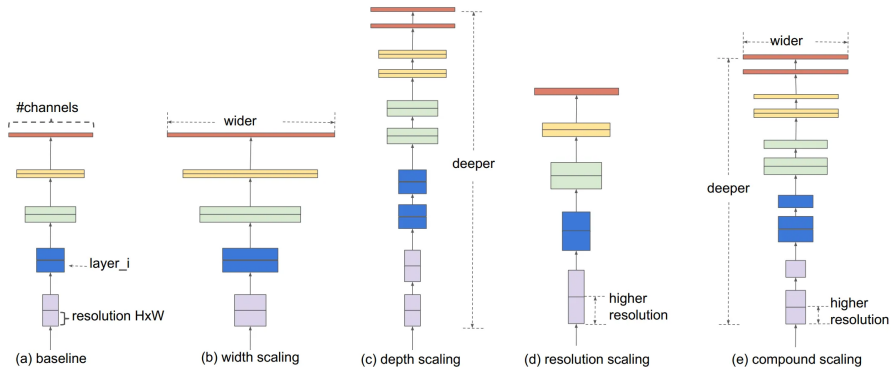


CNN Architectures: **EfficientNet**

- ▶ **Problem:** To improve accuracy, we can:
 - Increase the number of layers (**depth**)
 - Increase the number of neurons in each layer (**width**)
 - Use higher-resolution images (**resolution**)

But finding the right balance of these three was largely based on trial and error.

- ▶ **Solution:** EfficientNet introduced a **compound scaling** method—a mathematical formula to systematically find the optimal balance among **depth**, **width**, and **resolution**.



EfficientNet Scaling.

- ▶ **Compound Scaling** Uses a single **scaling coefficient** (ϕ) to control:
 - **Network Depth** (α^ϕ) $\rightarrow$ More layers
 - **Network Width** (β^ϕ) $\rightarrow$ More channels per layer
 - **Input Resolution** (γ^ϕ) $\rightarrow$ Larger input images
- ▶ The goal: find α, β, γ that balance accuracy & efficiency, then scale up optimally by increasing the global coefficient ϕ .

- ▶ EfficientNet optimizes depth, width, and resolution using this constraint:

$$\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

- ▶ **Why this equation?**

- Increasing **depth** (α) increases FLOPs **linearly**.
 - Increasing **width** (β) increases FLOPs **quadratically** (β^2).
 - Increasing **resolution** (γ) increases FLOPs **quadratically** (γ^2).
- ▶ To double total FLOPs, the three factors must be balanced together.

- ▶ The authors of EfficientNet searched for the best scaling factors on a small baseline model.
- ▶ They found:

$$\alpha = 1.2, \quad \beta = 1.1, \quad \gamma = 1.15$$

- The EfficientNet family (B0 to B7) is generated using:

$$\text{Depth} = 1.2^\phi, \quad \text{Width} = 1.1^\phi, \quad \text{Resolution} = 1.15^\phi$$

Model	ϕ	Depth (α^ϕ)	Width (β^ϕ)
B0	0	$1.2^0 = 1.0$	$1.1^0 = 1.0$
B1	1	$1.2^1 = 1.2$	$1.1^1 = 1.1$
B2	2	$1.2^2 = 1.44$	$1.1^2 = 1.21$
B3	3	$1.2^3 = 1.73$	$1.1^3 = 1.33$
B4	4	$1.2^4 = 2.07$	$1.1^4 = 1.46$
B5	5	$1.2^5 = 2.49$	$1.1^5 = 1.61$
B6	6	$1.2^6 = 2.99$	$1.1^6 = 1.77$
B7	7	$1.2^7 = 3.58$	$1.1^7 = 1.94$

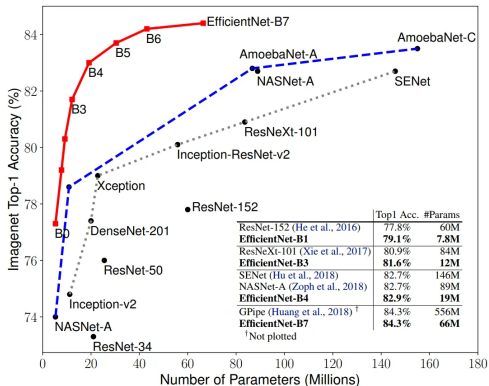
Scaling EfficientNet from B0 to B7

- We multiply these scaling factors by the baseline EfficientNet-B0 values and round to the nearest integer to get the new depth, width, and resolution for each model.

- ▶ EfficientNet models achieve state-of-the-art accuracy with significantly fewer parameters and FLOPs.

- ▶ EfficientNet models achieve state-of-the-art accuracy with significantly fewer parameters and FLOPs.
- ▶ EfficientNet-B7 reaches 84.4% Top-1 and 97.3% Top-5 accuracy on ImageNet.

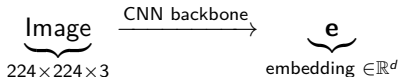
- ▶ EfficientNet models achieve state-of-the-art accuracy with significantly fewer parameters and FLOPs.
- ▶ EfficientNet-B7 reaches 84.4% Top-1 and 97.3% Top-5 accuracy on ImageNet.
- ▶ More efficient than previous CNN models—8.4x smaller and 6.1x faster than competitors.



EfficientNet Performance on Imagenet.

Pretrained CNNs: **Embeddings**

- ▶ A pretrained CNN has two parts:
 - **Feature extractor** (convolutional layers): learns visual patterns.
 - **Classification head** (final FC layers): maps features to class labels.
- ▶ If we **remove the classification head**, the output of the feature extractor is a *fixed-length vector* — the **embedding**.
- ▶ This vector is a compact numerical summary of the image's visual content.



- ▶ For ResNet-50: $d = 2048$. For MobileNet-v2: $d = 1280$.

- ▶ **Key property:** Images that look similar produce **nearby vectors** in embedding space.
- ▶ Two images of cats will have similar embeddings, even if their pixel values are very different.
- ▶ We can measure similarity between two embeddings using:

Cosine Similarity:

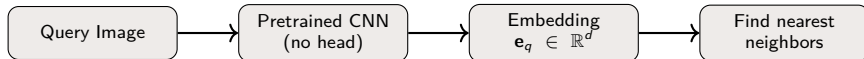
$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

Euclidean Distance:

$$d(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_2$$

- ▶ Cosine similarity ranges from -1 to 1 (higher = more similar).
- ▶ Euclidean distance: lower = more similar.

- **Image search / retrieval:** Given a query image, find the most visually similar images in a database.



- **Offline:** Compute embeddings for all images in the database and store them.
- **Online:** Compute the query embedding, find the k closest stored embeddings.
- **Used by:** Google Images, Pinterest, e-commerce visual search.

- ▶ Embeddings are what makes **transfer learning** possible.
- ▶ The pretrained feature extractor already “understands” visual patterns.
- ▶ To adapt to a new task, we just need to:
 - Use the embeddings as input features for a new classifier, or
 - Fine-tune the feature extractor on our new data.
- ▶ **Next:** How to do this systematically — Transfer Learning.

CNN: **Transfer Learning**

- ▶ **Transfer Learning** is the process of reusing a model trained on one task (e.g., ImageNet classification) for a different but related task (e.g., medical image analysis).
- ▶ **Why use it?**
 - Saves training time and computational resources.
 - Achieves good performance with limited data.
- ▶ **Types:**
 - **Feature extraction:** Freeze convolutional layers and only train the final classifier.
 - **Fine-tuning:** Unfreeze some or all layers and retrain on the new task.

- ▶ **Example:** Load a pretrained ResNet50 from PyTorch.
- ▶ Replace the last (classification) layer to match your number of classes.
- ▶ Optionally unfreeze the last few layers and retrain (fine-tuning).

Use cases:

- ▶ Medical imaging
- ▶ Satellite imagery
- ▶ Small business datasets with limited data

```
import torch
import torchvision.models as models
import torch.nn as nn

# Load pretrained ResNet50
model = models.resnet50(pretrained=True)

# Freeze all layers
for param in model.parameters():
    param.requires_grad = False

# Replace the final layer for 3 classes
num_fts = model.fc.in_features
model.fc = nn.Linear(num_fts, 3)

# Optionally unfreeze last few layers for fine-tuning
for param in list(model.parameters())[-10:]:
    param.requires_grad = True
```

- ▶ **Faster Training:** Pretrained models converge faster than training from scratch.
- ▶ **Better Performance:** Leverages learned features from large datasets.
- ▶ **Reduced Overfitting:** Especially useful when training data is limited.
- ▶ **Flexibility:** Can adapt to various tasks with minimal changes.

Considerations:

- ▶ Ensure the pretrained model is suitable for your task.
- ▶ Fine-tuning requires careful selection of layers to unfreeze.
- ▶ Monitor for overfitting, especially with small datasets.

Transfer Learning: **Fine-Tuning Strategies**

- ▶ **Freeze** the entire pretrained backbone.
- ▶ **Replace** the classification head with a new one for your task.
- ▶ Only the new head is trained — very fast, very few parameters.

When to use:

- ▶ Your dataset is **small** (hundreds to a few thousand images).
- ▶ Your task is **similar** to ImageNet (natural images, common objects).
- ▶ You have **limited compute** (e.g., no GPU).

Conv layers (frozen) → New FC head (trainable)

- ▶ **Unfreeze all** layers in the pretrained model.
- ▶ Train the entire network on your data with a **small learning rate**.
- ▶ The backbone adapts its features to your specific domain.

When to use:

- ▶ Your dataset is **large** (tens of thousands of images or more).
- ▶ Your task is **different** from ImageNet (e.g., X-rays, satellite images).
- ▶ You have **sufficient compute** (GPU recommended).

Conv layers (trainable, small LR) → New FC head (trainable)

- ▶ Start with feature extraction (all layers frozen).
- ▶ Progressively **unfreeze layers from the top** (deepest first).
- ▶ Train for a few epochs after each unfreezing step.

Why this works:

- ▶ Early layers learn **generic features** (edges, textures) — rarely need changing.
- ▶ Deeper layers learn **task-specific features** — benefit most from adaptation.
- ▶ Gradual unfreezing prevents catastrophic forgetting of pretrained knowledge.

When to use:

- ▶ Your dataset is **medium-sized**.
- ▶ You want a balance between speed and adaptation quality.

	Similar to ImageNet	Different from ImageNet
Small dataset	Feature extraction	Feature extraction + cautious fine-tuning
Large dataset	Fine-tune deeper layers	Full fine-tuning

- **Rule of thumb:** Start simple (feature extraction), add complexity only if needed.

Step 1: Load and Freeze

```
import torch
import torch.nn as nn
import torchvision.models as models

# Load pretrained ResNet-18
model = models.resnet18(weights='IMAGENET1K_V1')

# Freeze all backbone parameters
for param in model.parameters():
    param.requires_grad = False

# Replace classification head (ResNet's fc layer)
num_ftrs = model.fc.in_features    # 512 for ResNet-18
model.fc = nn.Sequential(
    nn.Linear(num_ftrs, 256),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(256, 5),    # 5 classes for our task
)
```

Step 2: Train Only the Head

```
device = torch.device('cuda' if
                      torch.cuda.is_available()
                      else 'cpu')
model = model.to(device)

# Only parameters with requires_grad=True are updated
optimizer = torch.optim.Adam(
    filter(lambda p: p.requires_grad,
           model.parameters()),
    lr=1e-3
)
criterion = nn.CrossEntropyLoss()

# Standard training loop (same as Day 7)
for epoch in range(10):
    model.train()
    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Step 3: Gradual Unfreezing (Optional)

```
# After training the head, unfreeze the last block
for param in model.layer4.parameters():
    param.requires_grad = True

# Use a SMALLER learning rate for pretrained layers
optimizer = torch.optim.Adam([
    {'params': model.layer4.parameters(),
     'lr': 1e-5},
    {'params': model.fc.parameters(),
     'lr': 1e-3},
])

# Continue training for a few more epochs
for epoch in range(5):
    model.train()
    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

- ▶ **Pick a solid architecture:** Start with proven models like ResNet, MobileNet, EfficientNet.
- ▶ **Use pretrained weights:** Leverage features learned on ImageNet.
- ▶ **Extract embeddings:** Use the backbone as a feature extractor for search, retrieval, or downstream tasks.
- ▶ **Transfer learning:** Replace the head and train on your data.
- ▶ **Fine-tune when needed:** Gradually unfreeze layers for domain-specific tasks.
- ▶ **Apply best practices from Day 8:** Data augmentation, dropout, batch normalization, learning rate schedules.

Approach	When
Train from scratch	Very large dataset, unique domain, enough compute
Feature extraction	Small dataset, similar to ImageNet, limited compute
Fine-tuning	Medium-to-large dataset, different domain
Embeddings	Image search, retrieval, similarity tasks

Credits

KAUST Academy

King Abdullah University of Science and Technology

CNN architecture content from KAUST Academy Computer Vision materials

KAUST–Mawhiba IOAI 2026 Training Program